

Nama : Khohar Muhamad Fatahurrohman

NIM : 254107020021

Kelas : 1G TI

Tanggal : 10/03/2026

5.2.3 Pertanyaan (Faktorial)

1. Perbedaan kode if dan else pada algoritma Divide and Conquer:

if ($n == 1$) merupakan base case atau kondisi berhenti dari rekursi. Jika nilai sudah 1 maka fungsi langsung mengembalikan nilai 1.

else merupakan proses rekursif yang memanggil fungsi faktorialDC($n-1$) sehingga perhitungan dilakukan berulang hingga mencapai base case.

2. Apakah perulangan for dapat diganti?

Ya, perulangan pada faktorialBF() bisa diganti dengan while atau do-while.

Contoh menggunakan while:

```
int faktorialBF(int n){ int fakto=1; int i=1; while(i<=n){ fakto=fakto*i; i++; } return fakto; }
```

3. Perbedaan fakto *= i dan int fakto = n * faktorialDC($n-1$):

- fakto *= i digunakan pada metode Brute Force dengan perulangan.
- int fakto = n * faktorialDC($n-1$) digunakan pada metode Divide and Conquer menggunakan rekursi.

4. Kesimpulan perbedaan faktorialBF() dan faktorialDC():

- Brute Force menggunakan perulangan.
- Divide and Conquer menggunakan rekursi.
- Brute Force menghitung secara langsung sedangkan Divide and Conquer membagi masalah menjadi bagian lebih kecil.

5.3.3 Pertanyaan (Pangkat)

1. Perbedaan pangkatBF() dan pangkatDC():

- pangkatBF() menggunakan perulangan untuk mengalikan angka sebanyak nilai pangkat.
- pangkatDC() menggunakan rekursi dengan metode divide and conquer.

2. Apakah tahap combine ada pada kode?

Ya, tahap combine terdapat pada proses penggabungan hasil rekursi seperti:

```
return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
```

3. Apakah method masih relevan memiliki parameter?

Ya, karena membuat method lebih fleksibel. Namun method juga bisa dibuat tanpa parameter jika menggunakan atribut class.

4. Kesimpulan cara kerja pangkatBF() dan pangkatDC():

- pangkatBF menghitung dengan perulangan.
- pangkatDC membagi masalah pangkat menjadi bagian lebih kecil kemudian menggabungkannya kembali.

5.4.3 Pertanyaan (Sum Array)

1. Mengapa dibutuhkan variabel mid?

Variabel mid digunakan untuk membagi array menjadi dua bagian (kiri dan kanan).

2. Fungsi statement:

```
double lsum = totalDC(arr, l, mid);
```

```
double rsum = totalDC(arr, mid+1, r);
```

Untuk menghitung jumlah bagian kiri dan kanan array.

3. Mengapa perlu lsum + rsum?

Karena hasil total array merupakan gabungan dari total bagian kiri dan kanan.

4. Base case dari totalDC():

```
if(l == r) return arr[l];
```

Artinya jika hanya satu elemen maka langsung dikembalikan.

5. Kesimpulan cara kerja totalDC():

Algoritma membagi array menjadi dua bagian (divide), menghitung masing-masing bagian secara rekursif (conquer), lalu menjumlahkan hasilnya (combine).

5.4 Latihan Praktikum

- a) Nilai UTS tertinggi: 92 (Eko)
- b) Nilai UTS terendah: 76 (Dian)
- c) Rata-rata nilai UAS:

Total UAS = 683

Rata-rata = $683 / 8 = 85.375$